East West University

Department of CSE

PROJECT REPORT

Course Code : CSE325 **Course Title:** Operating Systems	
Project No: 01 **Group No: 01**	
Project Name: Seeking Tutor Problem	
Semester and Year: Fall 2023	**Section: 01**
Name of Student: **1. Ratry** **2. Rifa Azad** **3. Abir Hasan** **Student Id:** **1. 2020-3-60-002** **2. 2022-1-60-265** **3. 2021-1-60-035**	**Dr. Md. Nawab Yousuf Ali** **Professor** **Department of Computer Science & Engineering**
Date of Report Submission: 26th December, 2023	

Table of Contents

Index	Topic	Page no.
1	Problem Statement	3
2	Approach	3
3	Key Elements Analysis of the Codebase	3 - 4
4	Coding Challenges and Limitations	4 - 5
5	Program Code	5 - 9
7	Program Output	10 - 15

Problem Statement

"**Seeking Tutor Problem**" is a program made for assisting students in finding suitable tutors for their academics.

The program simulates a tutoring environment where students seek help from tutors. The number of students, tutors, chairs, and the maximum number of times a student can ask for help are configurable by the user.

Semaphores are used to synchronize access to shared resources, such as chairs and the coordinator, students and the coordinator, students and tutors.

Approach

We implemented a solution that synchronizes the activities of the tutors and the students by using POSIX threads, mutex locks, and semaphores.

The program initializes various variables, semaphores, and arrays to keep track of the state of the simulation, including the number of chairs, students, tutors, and a limit on the number of times a student can ask for help. There are arrays to store priorities, visited queues, and other information related to students and tutors.

The code defines three thread functions: `student_t`, `tutor_t`, and `coordinator_t`. These functions simulate the behavior of students, tutors, and the coordinator, respectively. Each thread executes an infinite loop representing the continuous nature of the tutoring program.

A `start_simulation` function is implemented to initialize the simulation, including user input for the number of students, tutors, chairs, and the help limit. It also initializes semaphores and creates threads for students, tutors, and the coordinator.

The main function of the program presents a menu to the user with options to either start the simulation or terminate the program. It utilizes a loop to repeatedly prompt the user for input and execute the corresponding functionality until the user does not wish to terminate the whole program.

Key Elements Analysis of the Codebase

- **Student Threads (`student_t()`):** Each student thread runs in a loop for the given input students, attempting to acquire a chair and then requesting

help from a tutor. Students have a limit on the number of times they can seek help.

- **Tutor Threads (`tutor_t()`):** Each tutor thread runs in an in loop for the given input students, selecting a student based on priority and time of arrival. Tutors then provide help to the selected student.

- **Coordinator Thread (`coordinator_t()`):** The coordinator thread manages the queue of students waiting for help. It updates priorities and signals tutors when a student is available for assistance.

- **Simulation (`start_simulation()`):** The simulation is initiated by the user, who provides the number of students, tutors, chairs, and the help limit. Threads for students, tutors, and the coordinator are created and run concurrently.

- **Main Function (`main()`):** The main function presents a menu to the user, allowing them to start the simulation or terminate the program based on their choice.

Coding Challenges and Limitations

Problems we faced during coding:

- At the initial execution, the code outputted only half of the expected results and abruptly halted midway. The program neither terminated nor displayed incorrect results, a situation attributed to a miscommunication between semaphores. Later on, we addressed this issue by enhancing the synchronization of the semaphores.
- It was another issue to find the unsynchronized semaphores in the threads due to the uses of `sem_wait(&semaphore)` & `sem_post(&semaphore)`. To solve the problem we need to re calculate the operations of *wait & signal operations* and was successfully able to run the program without any interruption.

Limitations of the code:

- The program currently has set limits for the maximum number of students, tutors, and chairs, denoted by MAX_STUDENT, MAX_TUTOR and MAX_CHAIR. Unfortunately, these predetermined limits (Macros) restrict the program's ability to efficiently handle larger scenarios, affecting its scalability for future large input data use.
- The program heavily relies on semaphores and mutex locks. If not managed appropriately, this could result in deadlocks. For example, if the

co-ordination between students waiting in the waiting room and those assigned to a tutor is not handled carefully, it could bring the entire program to a standstill. In such situation, no further progress would be made, and the program may neither terminate nor execute properly.

- In the user input section, the program is designed to correct a user's mistaken limitation for the total number of students, tutors, or chairs at once. However, if the user provides incorrect input again, the program currently doesn't verify it, allowing the execution to proceed. This may lead to potential errors during the program's execution. The program should be enhanced in a way that it can consistently validate user input to prevent potential issues in case of repeated incorrect entries.
- The program lacks the ability to dynamically adjust to changing conditions, such as an increasing number of students requiring help or increasing the number of tutors, rather than limited.

Program Code

```
#include<stdio.h>
#include<stdlib.h>
#include<semaphore.h>
#include<pthread.h>
#include<unistd.h>
#include<stdbool.h> // for true in while condition

#define MAX_STUDENT 100
#define MAX_TUTOR 10
#define MAX_CHAIR 25

      /*Initialiazations , Semaphores & Threads*/

int total_chair, total_student, total_tutor, help_limit;
int req_order = 0, helped_student = 0, occupied_chair = 0;

int priority[MAX_STUDENT], visited_q[MAX_STUDENT],
total_visit[MAX_STUDENT];
int tutor_ID[MAX_TUTOR], student_ID[MAX_STUDENT],
time_arrive[MAX_STUDENT];

sem_t student_s, coordinator_s, tutored_student[MAX_STUDENT],
mutex_lock;

//pthread_mutex_t Locked_seat, Locked_q, Locked_tutor_q;
```

```
	/*Thread Function Pointers*/

void *student_t(void *arg){
	int student_id = *(int *) arg;

	while(true){
		if(total_visit[student_id - 1] == help_limit){
			sem_wait(&mutex_lock);
			helped_student++;
			sem_post(&mutex_lock);
			printf("\n!!!Student - %d terminated (Help Limit
Reached))\n", student_id);
			if(helped_student == total_student)
				printf("\nCo-ordinator: All students have
received Help.\n");
			sem_post(&student_s);
			pthread_exit(NULL);
		}

		sem_wait(&mutex_lock);
		if(occupied_chair == total_chair){
			sem_post(&mutex_lock);
			continue;
		}
		occupied_chair++;
		req_order++;
		visited_q[student_id-1] = req_order;
		printf("\n||   Student %d   || Seated. Available Chairs:
%d", student_id, (total_chair-occupied_chair));
		sem_post(&mutex_lock);
		sem_post(&student_s);
		sem_wait(&tutored_student[student_id-1]);

		printf("\n||   Student %d   || received Help.",
student_id);
		sem_wait(&mutex_lock);
		total_visit[student_id - 1]++;
		printf("\n||  Student %d  || Priority: %d", student_id,
total_visit[student_id - 1]);
		sem_post(&mutex_lock);
	}
}

void *tutor_t(void *arg){
	int tutor_id = *(int *) arg;

	while(true){
		if(helped_student == total_student){
			sem_wait(&mutex_lock);
			printf("\n!!!Tutor - %d has finished Task.\n",
tutor_id);
			sem_post(&mutex_lock);
			pthread_exit(NULL);
		}
		int max_req = total_student * help_limit + 1;
		int max_priority = help_limit - 1;
```

```
            int student_id = -1;
            sem_wait(&coordinator_s);
            sem_wait(&mutex_lock);
            for(int i = 0; i < total_student; i++){
                  if(priority[i] > -1 && priority[i] <= max_priority){
                        if(time_arrive[i] < max_req){
                              max_priority = priority[i];
                              max_req = time_arrive[i];
                              student_id = student_ID[i];
                        }
                  }
            }
            if(student_id == -1){
                  sem_post(&mutex_lock);
                  continue;
            }
            priority[student_id - 1] = -1;
            time_arrive[student_id - 1] = -1;
            occupied_chair--;
            sem_post(&mutex_lock);
            sem_wait(&mutex_lock);
            printf("\n[   Tutor %d   ] tutoring Student-%d.",
tutor_id,student_id);
            sem_post(&mutex_lock);
            sem_post(&tutored_student[student_id - 1]);
      }
}

void *coordinator_t(void *arg){
      while(true){
            if(helped_student == total_student){
                  for (int i = 0; i < total_tutor; i++)
                        sem_post(&coordinator_s);
                  printf("\n!!!Co-ordinator terminated!!!\n");
                  pthread_exit(NULL);
            }

            sem_wait(&student_s);
            sem_wait(&mutex_lock);
            for (int i = 0; i < total_student; i++){
                  if(visited_q[i] > -1){
                        priority[i] = total_visit[i];
                        time_arrive[i] = visited_q[i];
                        printf("\n<<Co-ordinator>> Student- %d  of
Priority- %d is NEXT in LINE.", student_ID[i], total_visit[i]);
                        visited_q[i] = -1;
                        sem_post(&coordinator_s);
                  }
            }
            sem_post(&mutex_lock);
      }
}

void start_simulation(){

      printf("\n\t\t=====================================\n");
```

```
	printf("\t\t||  Welcome to Tutoring Program  ||\n");
	printf("\t\t=====================================\n");

	printf("\nEnter Total Number Of Students: ");
	scanf("%d", &total_student);

	if(total_student > MAX_STUDENT || total_student < 1){
		printf("Student Limit Exceeded (not more than 100 & non-
negative).\nEnter value again: ");
		scanf("%d", &total_student);
	}

	printf("Enter Total Number Of Tutors: ");
	scanf("%d", &total_tutor);

	if(total_tutor > MAX_TUTOR || total_tutor < 1){
		printf("Tutor Limit Exceeded (not more than 10 & non-
negative).\nEnter value again: ");
		scanf("%d", &total_tutor);
	}

	printf("Enter Total Number Of Chairs: ");
	scanf("%d", &total_chair);

	if(total_chair > MAX_CHAIR || total_chair < 1){
		printf("Chair Limit Exceeded (not more than 25 & less
than 2).\nEnter value again: ");
		scanf("%d", &total_chair);
	}

	printf("Maximum Number of help a student can ask for: ");
	scanf("%d", &help_limit);

		/*Semaphores Initialization*/
	sem_init(&student_s, 0, 0);
	sem_init(&coordinator_s, 0, 0);
	sem_init(&mutex_lock, 0, 1);

	for(int i = 0; i < total_student; i++){
		visited_q[i] = -1;
		priority[i] = -1;
		time_arrive[i] = -1;
		total_visit[i] = 0;
		student_ID[i] = i + 1;
		sem_init(&tutored_student[i], 0, 0);
	}

		/*Thread Declaration & Creation*/
	pthread_t student[MAX_STUDENT], tutor[MAX_TUTOR],
coordinator_t_id;

	for(int i = 0; i < total_student; i++)
		pthread_create(&student[i], NULL, student_t,
(void*)&student_ID[i]);

	for(int i = 0; i < total_tutor; i++){
```

```
            tutor_ID[i] = i + 1;
            pthread_create(&tutor[i], NULL, tutor_t,
(void*)&tutor_ID[i]);
      }

      pthread_create(&coordinator_t_id, NULL, coordinator_t, NULL);


      /*Thread Joining*/
      for(int i = 0; i < total_student; i++)
            pthread_join(student[i], NULL);

      for(int i = 0; i < total_tutor; i++)
            pthread_join(tutor[i], NULL);

      pthread_join(coordinator_t_id, NULL);
}

      /*MAIN FUNCTION*/
int main(){
      int menu;
      printf("\n\t::::::::::::::::::::::::::::::::::::::::::::::::::
::::::::::::::::\n");
      printf("\t\t\tProgramming Club Management System\n");
      printf("\t::::::::::::::::::::::::::::::::::::::::::::::::::
::::::::::::::\n");
      while(1){
            printf("\t..........................Option
Board..........................\n\n");

            printf("\t\t\t-----------------------------------\n");
            printf("\t\t\t  Program Simulation          1\n");
            printf("\t\t\t  Program Terminate           2\nYour
Choice: ");
            scanf("%d", &menu);

            switch(menu){
                  case 1:
                        start_simulation();
                        break;
                  case 2:
                        printf("\n\t\t^_^ Thank you for coming.
^_^\n");
                        exit(1);
                        break;
                  default:
                        printf("\n\t\t!!! Wrong Input. Try Again
!!!!\n");
                        break;
            }
      }
      return 0;
}
```

Program Output

```
        ::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
                        Programming Club Management System
        ::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
        ..........................Option Board..........................

                        ----------------------------------
                          Program Simulation            1
                          Program Terminate             2
Your Choice: 1

                 ======================================
                 ||      Welcome to Tutoring Program  ||
                 ======================================

Enter Total Number Of Students: 15
Enter Total Number Of Tutors: 6
Enter Total Number Of Chairs: 22
Maximum Number of help a student can ask for: 3

||   Student 5   || Seated. Available Chairs: 21
||   Student 1   || Seated. Available Chairs: 20
||   Student 6   || Seated. Available Chairs: 19
||   Student 7   || Seated. Available Chairs: 18
||   Student 9   || Seated. Available Chairs: 17
||   Student 8   || Seated. Available Chairs: 16
||   Student 10   || Seated. Available Chairs: 15
||   Student 4   || Seated. Available Chairs: 14
||   Student 2   || Seated. Available Chairs: 13
||   Student 11   || Seated. Available Chairs: 12
||   Student 3   || Seated. Available Chairs: 11
||   Student 14   || Seated. Available Chairs: 10
||   Student 15   || Seated. Available Chairs: 9
||   Student 12   || Seated. Available Chairs: 8
||   Student 13   || Seated. Available Chairs: 7
<<Co-ordinator>> Student- 1  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 2  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 3  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 4  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 5  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 6  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 7  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 8  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 9  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 10  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 11  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 12  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 13  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 14  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 15  of Priority- 0 is NEXT in LINE.
```

```
[   Tutor 6   ] tutoring Student-5.
||   Student 5   || received Help.
||  Student 5  || Priority: 1
||   Student 5   || Seated. Available Chairs: 8
[   Tutor 5   ] tutoring Student-6.
||   Student 6   || received Help.
[   Tutor 2   ] tutoring Student-7.
||   Student 7   || received Help.
||  Student 7  || Priority: 1
||   Student 7   || Seated. Available Chairs: 9
[   Tutor 2   ] tutoring Student-9.
||   Student 9   || received Help.
||  Student 9  || Priority: 1
||   Student 9   || Seated. Available Chairs: 9
[   Tutor 2   ] tutoring Student-8.
||   Student 8   || received Help.
||  Student 8  || Priority: 1
||   Student 8   || Seated. Available Chairs: 9
[   Tutor 2   ] tutoring Student-10.
||   Student 10   || received Help.
||  Student 10  || Priority: 1
||   Student 10   || Seated. Available Chairs: 9
[   Tutor 2   ] tutoring Student-4.
[   Tutor 2   ] tutoring Student-2.
[   Tutor 2   ] tutoring Student-11.
[   Tutor 2   ] tutoring Student-3.
||   Student 3   || received Help.
||  Student 3  || Priority: 1
||   Student 3   || Seated. Available Chairs: 12
[   Tutor 1   ] tutoring Student-14.
||   Student 14   || received Help.
||  Student 14  || Priority: 1
||   Student 14   || Seated. Available Chairs: 12
||  Student 6  || Priority: 1
||   Student 6   || Seated. Available Chairs: 11
||   Student 4   || received Help.
||  Student 4  || Priority: 1
||   Student 4   || Seated. Available Chairs: 10
||   Student 2   || received Help.
||  Student 2  || Priority: 1
||   Student 2   || Seated. Available Chairs: 9
||   Student 11   || received Help.
||  Student 11  || Priority: 1
||   Student 11   || Seated. Available Chairs: 8
[   Tutor 6   ] tutoring Student-15.
[   Tutor 5   ] tutoring Student-12.
<<Co-ordinator>> Student- 2  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 3  of Priority- 1 is NEXT in LINE.
||   Student 12   || received Help.
||   Student 15   || received Help.
<<Co-ordinator>> Student- 4  of Priority- 1 is NEXT in LINE.
```

```
<<Co-ordinator>> Student- 5  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 6  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 7  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 8  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 9  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 10  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 11  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 14  of Priority- 1 is NEXT in LINE.
[   Tutor 6   ] tutoring Student-13.
[   Tutor 6   ] tutoring Student-5.
||   Student 5   || received Help.
||  Student 5  || Priority: 2
||   Student 5   || Seated. Available Chairs: 11
[   Tutor 6   ] tutoring Student-7.
[   Tutor 6   ] tutoring Student-9.
[   Tutor 6   ] tutoring Student-8.
||   Student 8   || received Help.
||  Student 8  || Priority: 2
||   Student 8   || Seated. Available Chairs: 13
[   Tutor 5   ] tutoring Student-10.
[   Tutor 5   ] tutoring Student-3.
[   Tutor 5   ] tutoring Student-14.
||   Student 14   || received Help.
||  Student 14  || Priority: 2
||   Student 14   || Seated. Available Chairs: 15
[   Tutor 6   ] tutoring Student-6.
||   Student 6   || received Help.
||  Student 6  || Priority: 2
||   Student 6   || Seated. Available Chairs: 15
[   Tutor 6   ] tutoring Student-4.
||   Student 4   || received Help.
||   Student 7   || received Help.
||   Student 10   || received Help.
||   Student 9   || received Help.
||  Student 4  || Priority: 2
||   Student 4   || Seated. Available Chairs: 15
||  Student 15  || Priority: 1
||   Student 15   || Seated. Available Chairs: 14
||  Student 9  || Priority: 2
||   Student 9   || Seated. Available Chairs: 13
||  Student 10  || Priority: 2
||   Student 10   || Seated. Available Chairs: 12
||   Student 3   || received Help.
||  Student 3  || Priority: 2
||   Student 3   || Seated. Available Chairs: 11
||  Student 7  || Priority: 2
||   Student 7   || Seated. Available Chairs: 10
||  Student 12  || Priority: 1
||   Student 12   || Seated. Available Chairs: 9
||   Student 13   || received Help.
||  Student 13  || Priority: 1
```

```
||    Student 13    || Seated. Available Chairs: 8
[    Tutor 3    ] tutoring Student-2.
[    Tutor 4    ] tutoring Student-1.
||    Student 1    || received Help.
||   Student 1   || Priority: 1
||    Student 1    || Seated. Available Chairs: 8
[    Tutor 2    ] tutoring Student-11.
||    Student 2    || received Help.
||   Student 2   || Priority: 2
||    Student 2    || Seated. Available Chairs: 8
||    Student 11    || received Help.
||   Student 11   || Priority: 2
||    Student 11    || Seated. Available Chairs: 7
<<Co-ordinator>> Student- 1  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 2  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 3  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 4  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 5  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 6  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 7  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 8  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 9  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 10  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 11  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 12  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 13  of Priority- 1 is NEXT in LINE.
<<Co-ordinator>> Student- 14  of Priority- 2 is NEXT in LINE.
<<Co-ordinator>> Student- 15  of Priority- 1 is NEXT in LINE.
[    Tutor 5    ] tutoring Student-15.
[    Tutor 5    ] tutoring Student-12.
[    Tutor 5    ] tutoring Student-13.
[    Tutor 5    ] tutoring Student-1.
||    Student 1    || received Help.
||    Student 15    || received Help.
[    Tutor 4    ] tutoring Student-5.
[    Tutor 4    ] tutoring Student-8.
[    Tutor 4    ] tutoring Student-14.
[    Tutor 4    ] tutoring Student-6.
[    Tutor 4    ] tutoring Student-4.
||    Student 4    || received Help.
||   Student 4   || Priority: 3
!!!Student - 4 terminated (Help Limit Reached))

||    Student 5    || received Help.
||   Student 5   || Priority: 3
!!!Student - 5 terminated (Help Limit Reached))

||    Student 8    || received Help.
||    Student 14    || received Help.
||    Student 6    || received Help.
||   Student 1   || Priority: 2
```

```
||    Student 12    || Seated. Available Chairs: 18
<<Co-ordinator>> Student- 12  of Priority- 2 is NEXT in LINE.
[    Tutor 6    ] tutoring Student-1.
||    Student 1    || received Help.
||   Student 1   || Priority: 3
!!!Student - 1 terminated (Help Limit Reached))

[    Tutor 3    ] tutoring Student-15.
||    Student 15    || received Help.
||   Student 15   || Priority: 3
!!!Student - 15 terminated (Help Limit Reached))

[    Tutor 2    ] tutoring Student-13.
||    Student 13    || received Help.
||   Student 13   || Priority: 3
!!!Student - 13 terminated (Help Limit Reached))

[    Tutor 1    ] tutoring Student-12.
||    Student 12    || received Help.
||   Student 12   || Priority: 3
!!!Student - 12 terminated (Help Limit Reached))

||    Student 9    || received Help.
||   Student 9   || Priority: 3
!!!Student - 9 terminated (Help Limit Reached))

||    Student 11    || received Help.
||   Student 11   || Priority: 3
!!!Student - 11 terminated (Help Limit Reached))

Co-ordinator: All students have received Help.

!!!Co-ordinator terminated!!!

!!!Tutor - 6 has finished Task.

!!!Tutor - 4 has finished Task.

!!!Tutor - 5 has finished Task.

!!!Tutor - 2 has finished Task.

!!!Tutor - 3 has finished Task.

!!!Tutor - 1 has finished Task.
        ..........................Option Board..........................

                        ----------------------------------
                          Program Simulation            1
                          Program Terminate             2
Your Choice: 2

                ^_^ Thank you for coming. ^_^
naomi@Kieren-VirtualBox:~/Desktop/Project$ sss
```

Output with Invalid Choice by User

```
        ::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
                        Programming Club Management System
        ::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
        ..........................Option Board..........................

                        -----------------------------------
                          Program Simulation            1
                          Program Terminate             2
Your Choice: 1

                ======================================
                ||      Welcome to Tutoring Program  ||
                ======================================

Enter Total Number Of Students: 250
Student Limit Exceeded (not more than 100 & non-negative).
Enter value again: 95
Enter Total Number Of Tutors: -2
Tutor Limit Exceeded (not more than 10 & non-negative).
Enter value again: 3
Enter Total Number Of Chairs: 0
Chair Limit Exceeded (not more than 25 & less than 2).
Enter value again: 6
Maximum Number of help a student can ask for: 2

||   Student 1   || Seated. Available Chairs: 5
```

Output with Insufficient Chairs

```
                ======================================
                ||      Welcome to Tutoring Program  ||
                ======================================

Enter Total Number Of Students: 10
Enter Total Number Of Tutors: 4
Enter Total Number Of Chairs: 6
Maximum Number of help a student can ask for: 2

||   Student 1   || Seated. Available Chairs: 5
||   Student 4   || Seated. Available Chairs: 4
||   Student 3   || Seated. Available Chairs: 3
||   Student 2   || Seated. Available Chairs: 2
||   Student 7   || Seated. Available Chairs: 1
<<Co-ordinator>> Student- 1  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 2  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 3  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 4  of Priority- 0 is NEXT in LINE.
<<Co-ordinator>> Student- 7  of Priority- 0 is NEXT in LINE.
||   Student 5   || Seated. Available Chairs: 0
```